Texture Aware Face Image Protection Using SVD and Expectation over Transformation

Patel Krishiv Anilkumar, Kethavath Eshwar, Mehul Krishna
Department of Computer Science and Engineering
Indian Institute of Information Technology Vadodara
International Campus Diu, India

***Abstract*—We implement a face-image protection pipeline that adds a small, image-dependent perturbation before an image is passed to a facial attribute editor.The method uses the two lowest singular directions of each RGB patch to guide optimization, together with a fixed channel-wise color offset.A second variant optimizes the expected output divergence across sampled image transformations.Both variants are evaluated using a pretrained StarGAN generator on one uploaded face image. The saved run gives input PSNR values of 40.05 dB and 40.25 dB for TAA and TAA-EOT, with SSIM near 0.96.Their generator-output mean squared errors are 0.019453 and 0.019917 in the normalized range. Actual JPEG compression at quality 70 reduces the TAA-EOT output error to 0.002371.These results demonstrate a working white-box prototype with good input fidelity, while also showing that broader evaluation and stronger compression robustness remain necessary.**

***Index Terms*—Adversarial perturbation,facial manipulation,singular value decomposition, StarGAN, expectation over transformation.**

I. Introduction

Facial attribute editors can modify hair color, apparent age, and other characteristics of a face while retaining much of its original appearance.Such tools create a practical motivation for protecting a source image before it is shared. In this project, protection means changing the input slightly so that a known editor produces an output that differs from the output it would produce on the clean image.

This objective follows proactive disruption of image manipulation [1]. The protected photograph should remain useful to its owner, while the downstream generator should respond differently. The implementation therefore measures both input fidelity and generator-output divergence. Output divergence is a proxy for disruption; it does not by itself prove that identity preservation or a requested attribute edit has failed.

We study a texture-guided variant named TAA in the notebook and its transformation-aware extension, TAA-EOT.The project implements preprocessing, patchwise singular value decomposition (SVD),constrained gradient updates,sampled transformations, quantitative evaluation,and image export.Its present scope is a single-image demonstration against one known generator and target condition.

II. Background And Project Scope

A. Conditional facial attribute editing

StarGAN uses a single conditional generator for translation across multiple image domains [2].In our experiment, the generator receives a face tensor and a five-element attribute vector.The official pretrained CelebA checkpoint is loaded successfully in the supplied run.The generator is frozen;only the input perturbation is optimized.

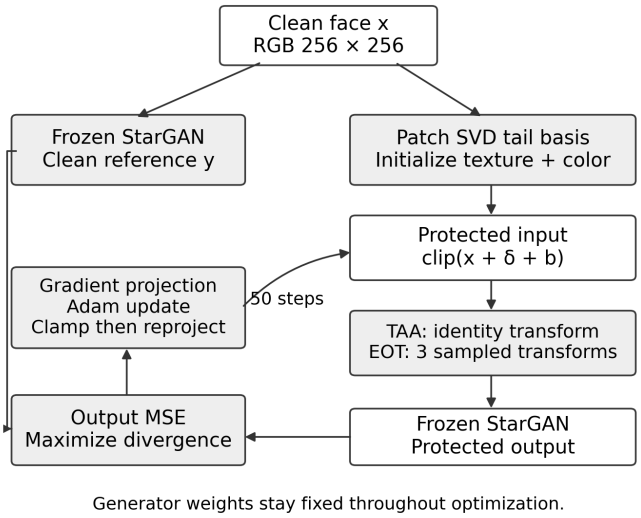


Fig. 1. Implemented pipeline. The clean reference is fixed. TAA uses the protected input directly;TAA-EOT averages three transformed evaluations per update.

B. Texture guidance and transformation sampling

SVD separates a patch into singular components ordered by their energy.The implementation uses a small subspace constructed from the two smallest singular directions. This is an image-adaptive matrix decomposition, rather than an explicit Fourier or DCT frequency restriction.Expectation over transformation (EOT) [3] adds stochastic preprocessing during optimization to seek disruption that survives changes to the input.

The report documents the executed notebook and its stored outputs.It does not claim universal protection across deepfake systems, a new generator architecture, or a statistically established improvement from EOT.

III. Protection Method

A. Image and generator representation

Let x be an RGB image with shape $1 \times 3 \times 256 \times 256$ in the range $[-1, 1]$, and let c denote the target condition. The clean reference is $y_{\text{ref}} = G(x, c)$.A learned texture perturbation δ_{t} and a fixed color term b form the protected image:

$$x_{\text{p}} = \text{clip}(x + \delta_{\text{t}} + b, -1, 1) \tag{1}$$

The target vector is [0, 1, 0, 0, 0] in the attribute order Black_Hair, Blond_Hair, Brown_Hair,Male, and Young. Consequently, the experiment sets Blond_Hair to one and the other four conditions to zero.It does not preserve inferred source attributes automatically.

B. Patchwise SVD and initialization

The image is divided into an 8×8 grid, yielding 64 spatial patches of 32×32 pixels. SVD is performed separately on each color channel of every patch. For a patch P,the decomposition and its retained tail component are:

$$P = U\Sigma V^{\mathsf{T}} \tag{2}$$

$$P_\tau = U_\tau \Sigma_\tau V_\tau^{\mathsf{T}} \tag{3}$$

U_τ and V_τ contain the last two columns of the left and right singular-vector matrices, and Σ_τ contains the corresponding two singular values.Each spatial patch thus has three independent channel decompositions.The retained tail reconstruction is assembled into a full-size tensor, normalized by its maximum absolute value, and multiplied by an initialization amplitude of 0.05.

The bases are computed from the clean image without backpropagating through svd. This avoids differentiating potentially unstable singular vectors and keeps the optimization subspace fixed. The code calls the SVD utility twice during setup, once to obtain the bases and once inside initialization; the decomposition is not repeated inside the 50-step loop.

C. Fixed color component

A seeded Gaussian sample supplies one scalar for each RGB channel. Each scalar has nominal standard deviation 0.01 and is clipped to $[-0.01, 0.01]$.It is broadcast over the image and kept fixed. The resulting perturbation combines a spatial texture component with a small global color offset.

D. Projected optimization

For any patch-shaped update A,the tail-space projection is:

$$\Pi_\tau(A) = U_\tau \big(U_\tau^{\mathsf{T}} A V_\tau\big) V_\tau^{\mathsf{T}} \tag{4}$$

This projection retains the span of all pairwise products between the two selected left and right directions.It therefore allows four coefficients per channel and patch;it is not limited to changing two singular values. The loss gradient is projected into this subspace before an Adam update [4].The updated texture tensor is then clipped componentwise and projected again.

The nominal texture clamp is $\epsilon = 0.05$ in $[-1, 1]$, equivalent to about 6.38/255 in [0, 1]. Because the projection follows the clamp and the color term is added separately, the current implementation does not certify that the complete final perturbation obeys the same infinity-norm bound. Final clipping guarantees valid image values.

E. Output divergence objectives

TAA maximizes the mean squared error between the protected and clean generator outputs.TAA-EOT averages this same distance over S=3 transformations sampled independently at each step:

$$J_{\text{EOT}} = \frac{1}{S} \sum_{s=1}^{S} \ell_s \tag{5}$$

$$\ell_s = \text{MSE}\big(G(T_s(x_{\text{p}}), c), y_{\text{ref}}\big) \tag{6}$$

Adam minimizes the negative of this objective for 50 steps at learning rate 0.01. The generator parameters require no gradients, but gradients still pass through the generator to the input. TAA omits the random transform and otherwise uses the same optimization procedure.

F. Operational sequence

Each run first computes the clean generator output and initializes the texture tensor.Every iteration constructs the protected input,evaluates one TAA loss or three sampled EOT losses, backpropagates the negative mean loss,projects the texture gradient,applies Adam, and clips then reprojects the iterate. The final protected image is evaluated without a transformation, independently of the last logged training loss.

The EOT reference in (6) is the untransformed clean output. Thus the current objective includes the effect of preprocessing as well as the perturbation.A matched transformed clean reference is an important next step for isolating protection-induced divergence.

IV. Implementation And Experimental Setup

A. Runtime and model

The executed notebook uses Google Colab with a Tesla T4 GPU. Python, NumPy, and PyTorch seeds are set to 42. Extra packages include LPIPS,scikit-image,Pillow,and pandas;the notebook retains Colab's PyTorch and CUDA stack. Its outputs confirm loading the official StarGAN 256×256 checkpoint, 200000-G.ckpt, rather than the untrained fallback.

The generator has 8,430,528 parameters. It uses an initial convolution conditioned on five attribute channels, two downsampling blocks,six residual blocks,two upsampling blocks, and a final tanh layer. Instance normalization is used in the feature blocks. The generator is placed in evaluation mode and its weights are frozen.

TABLE I
CONFIGURATION OF THE STORED SINGLE IMAGE RUN

Setting	Value
Image dimensions	256 × 256 RGB
Patch grid and patch size	8 × 8; 32 × 32
Tail rank per channel	2
SVD initialization amplitude	0.05
Nominal texture clamp ϵ	0.05 in $[-1, 1]$
Color noise standard deviation	0.01; clipped to ± 0.01
Optimization steps	50
Optimizer and learning rate	Adam; 0.01
EOT samples per step	3
Target condition	[0, 1, 0, 0, 0]
Random seed	42
Images evaluated	1; batch mode disabled

TABLE II
TRAINING TRANSFORMATION DISTRIBUTION

Mode	Implementation
Resize	Bilinear resize to a random 224–252 square, then back to 256
Blur surrogate	3 × 3 or 5 × 5 average pooling, stride 1
Intensity scaling	Multiply the normalized tensor by a factor in 0.92–1.08
Quantization	64, 96, or 128 levels with a straight-through gradient
Gaussian noise	Standard deviation sampled from 0.002–0.01

B. Source image and preprocessing

The input is the uploaded photograph Krishiv-Photo-Sept-2026.jpg. The preprocessing applies EXIF orientation correction, converts to RGB,takes a centered square crop, resizes using Lanczos interpolation, and maps pixel values from [0, 1] to $[-1, 1]$.No face detector or landmark alignment step is included.

The optional batch path supports up to 20 images and per-image, mean, and standard-deviation exports. In the supplied run, RUN_BATCH is false. The present results are therefore not dataset averages and have no measured confidence intervals.

C. Transformations during EOT

Each EOT sample chooses one of five modes uniformly. The corresponding settings are summarized in Table II. The quantization mode uses a straight-through estimator so that the discrete forward operation still permits input-gradient optimization.

The average-pooling blur surrogate is distinct from the Gaussian blur used at evaluation. Likewise, level quantization does not model JPEG's block DCT, quantization tables, or chroma processing. JPEG robustness is checked separately using real image encoding and decoding.

D. Evaluation metrics

Input PSNR and SSIM [5] compare clean and protected images in [0, 1]. LPIPS with an AlexNet backbone [6] uses the native $[-1, 1]$ tensors. Higher input PSNR and SSIM, together with lower LPIPS, indicate closer input fidelity. Input L1, Frobenius norm, and root mean squared change are also recorded.

TABLE III
INPUT FIDELITY AND GENERATOR OUTPUT DIVERGENCE

Metric	TAA	TAA-EOT
Input PSNR (dB) ↑	40.053446	40.248095
Input SSIM ↑	0.960040	0.961766
Input LPIPS ↓	0.001447	0.001497
Input L1 ↓	0.010992	0.010750
Input RMS change ↓	0.019877	0.019437
Output MSE ↑	0.019453	0.019917
Output PSNR (dB) ↓	23.130664	23.028409

Output MSE is computed directly on generator tensors in $[-1, 1]$. With N = 3 × 256 × 256 elements, its definition and the equivalent [0, 1] output PSNR are:

$$\mathrm{MSE}_{\mathrm{out}} = \frac{\|a - b\|_F^2}{N} \tag{7}$$

$$\mathrm{PSNR}_{\mathrm{out}} = 10 \log_{10}\left(\frac{4}{\mathrm{MSE}_{\mathrm{out}}}\right) \tag{8}$$

Higher output MSE and lower output PSNR indicate greater divergence from the clean edit. These measures are not face-recognition accuracy, attribute-classification accuracy, or a deepfake detection score.

E. Exported artifacts

The notebook writes protected PNG images, a real JPEG at quality 70, the perturbation tensor, configuration text, metric CSV files, a comparison figure, an optimization curve, and a ZIP bundle. Its final checks confirm these paths exist. PNG-labeled evaluation uses the in-memory protected tensor; an actual PNG reload is not part of the stored robustness test.

V. Results And Analysis

A. Input fidelity and untransformed disruption

Both protected inputs remain visually close to the original in Fig. 2. Their PSNR values exceed 40 dB and their SSIM values remain near 0.96. The small LPIPS values support this input-fidelity observation, although no human perceptual study has been conducted.

Input L1 and RMS changes in Table III are measured in $[-1, 1]$. They should not be interpreted as [0, 1] pixel errors without dividing by two. The figure is taken directly from the executed notebook; no synthetic result images are substituted.

B. Comparing TAA and TAA-EOT

For the untransformed protected input, EOT increases output MSE by 0.000464, approximately 2.39% relative to TAA. It also increases input PSNR by 0.194649 dB and SSIM by 0.001726. LPIPS is slightly higher for TAA-EOT. These small differences describe this specific run; they do not establish a consistent advantage across images or random seeds.

The clean StarGAN output already contains visible editing artifacts. Protected inputs introduce additional local changes, especially around the face. The output-difference map displays where the generator response changes relative to the same clean-input reference. We interpret this as measurable output



Fig. 2. Original stored notebook comparison. Top row: clean face, TAA input, TAA-EOT input, and absolute input difference amplified 20 times. Bottom row: StarGAN outputs for clean, TAA, and TAA-EOT inputs, followed by the mean absolute output difference map. Display amplification is used only to reveal the perturbation.

TABLE IV
TAA EOT DIVERGENCE AFTER PREPROCESSING

Transformation	**Output MSE**	**PSNR (dB)**
Untransformed	0.019917	23.028409
JPEG quality 90	0.002163	32.669926
JPEG quality 70	0.002371	32.271384
JPEG quality 50	0.002933	31.347045
Resize 256→224→256	0.002814	31.527253
Gaussian blur radius 1.0	0.004487	29.501230

disruption, rather than evidence that every facial manipulation becomes unusable.

C. Reading the optimization traces

The final untransformed EOT MSE is 0.019917, whereas the last logged EOT training objective is 0.014615. The two are evaluated under different inputs: the log averages random transformations before the final update, while the final metric uses the completed protected image with no transformation. The difference is therefore expected and should not be presented as an evaluation inconsistency.

D. Real preprocessing and compression

Table IV evaluates the TAA-EOT input after actual JPEG round trips, Lanczos downsampling and restoration, or Gaussian blur. All results use the same untransformed clean generator output as the reference. The first row is the in-memory input labeled PNG in the notebook.

JPEG quality 70 retains only about 11.90% of the untransformed output MSE, an 88.10% reduction in the measured divergence. Gaussian blur retains approximately 22.53%. These



Fig. 3. Stored actual JPEG quality 70 test. Left: StarGAN on the clean input. Right: StarGAN on the decoded protected JPEG. Output MSE is 0.002371.

observations show that the present perturbation is vulnerable to common image processing despite the EOT training.

The JPEG results are not monotonic with quality: quality 50 has a larger output MSE than quality 90. Lower quality also modifies the underlying image more strongly, and the current reference does not subtract that effect. Consequently, larger post-compression MSE does not necessarily mean better survival of the adversarial component.

A matched control would evaluate a transformed clean image alongside each transformed protected image. Robustness of TAA under these same tests is not recorded, so Table IV cannot establish that EOT improves transformation robustness over TAA.

E. Optimization behavior

TAA increases from approximately 0.000082 at step one to 0.019455 at step 50 and approaches a plateau. The EOT

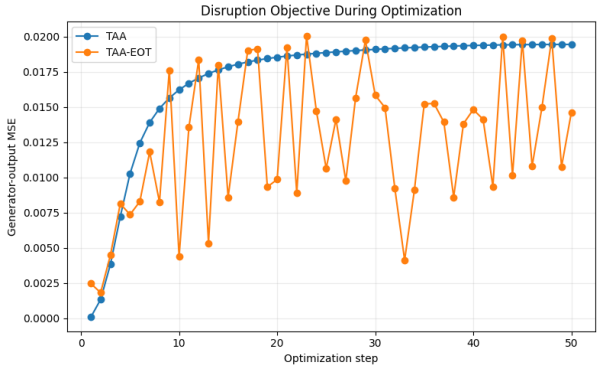


Fig. 4. Original 50-step objective traces. TAA evaluates a fixed input condition, while TAA-EOT averages three new transformation samples at each step. Their training objectives are not directly equivalent.

trace fluctuates because its sampled preprocessing changes between steps. Such fluctuations do not by themselves show divergence of the optimizer. More samples or a fixed validation transformation set could make progress easier to assess.

VI. Limitations and Validation

A. Evidence available in this implementation

The experiment covers one uploaded portrait, one generator checkpoint, and one target vector. There is no test-set aggregation, cross-model transfer study, repeated-seed comparison, or downstream attribute-success evaluation. A clean, untransformed reference is used even when the protected image is transformed, which mixes preprocessing effects with protection effects.

Input metrics are calculated on floating-point protected tensors before export. Protection must also be assessed after 8-bit encoding and decoding. The code does not record the maximum absolute final perturbation, and clipping followed by subspace projection does not enforce the intersection of both constraints exactly.

B. Auxiliary baseline consistency

The notebook labels an additional method Vanilla-PGD, but the function applies Adam updates with a pixel clamp. Its perturbation starts at exactly zero, and its squared output-distance objective has zero value and zero gradient at that starting point for the deterministic frozen generator. A nonzero saved result cannot be explained by that code path alone.

The stored baseline row is therefore excluded from performance claims. A fresh run with a documented nonzero initialization and an explicitly defined projected-gradient baseline [7] is needed before comparing texture guidance with unconstrained pixel-space optimization.

C. Steps toward a stronger evaluation

A first extension should compare each transformed protected input against the identically transformed clean input. Both TAA and TAA-EOT should be evaluated under the same resize, blur, and JPEG settings. A randomized pixel-space baseline should use the same generator, number of optimization steps, and a verified bound on the complete perturbation.

A second extension should enable batch evaluation on a documented face-image set and report per-image values, aggregate statistics, and repeated-seed variability. Target-condition sweeps should preserve source attributes where appropriate. Transfer tests on additional facial editors would determine whether protection depends strongly on this specific checkpoint.

To pursue a research contribution, compression-aware optimization could incorporate a differentiable approximation to actual JPEG and validate it with real codecs. An ablation should separate SVD initialization, subspace projection, the color term, and EOT. Semantic attribute-success and identity-similarity measures would complement pixel divergence. These are proposed experiments, not results of the present notebook.

VII. Conclusion

The implementation demonstrates a complete workflow for generating texture-guided protective perturbations and evaluating their effect on a pretrained facial attribute editor. TAA and TAA-EOT preserve the visual appearance of the tested input and produce measurable changes in the StarGAN output. The EOT variant shows a small untransformed divergence increase in the saved run.

Real JPEG compression and blur substantially weaken the measured effect. The current evidence therefore supports an implementation-stage prototype, with compression robustness, valid baseline comparisons, and larger-scale testing as the next priorities. The experimental figures and metrics in this report are reproduced from the supplied executed notebook.

References

[1] N. Ruiz, S. A. Bargal, and S. Sclaroff, “Disrupting Deepfakes: Adversarial Attacks Against Conditional Image Translation Networks and Facial Manipulation Systems,” arXiv:2003.01279, 2020. Available: https://arxiv.org/abs/2003.01279

[2] Y. Choi, M. Choi, M. Kim, J.-W. Ha, S. Kim, and J. Choo, “StarGAN: Unified Generative Adversarial Networks for Multi-Domain Image-to-Image Translation,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2018, pp. 8789–8797. doi: 10.1109/CVPR.2018.00916.

[3] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, “Synthesizing Robust Adversarial Examples,” in Proc. 35th Int. Conf. Machine Learning, vol. 80, 2018, pp. 284–293. Available: https://proceedings.mlr.press/v80/athalye18b.html

[4] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in Proc. Int. Conf. Learning Representations, 2015. Available: https://arxiv.org/abs/1412.6980

[5] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image Quality Assessment: From Error Visibility to Structural Similarity,” IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, Apr. 2004. doi: 10.1109/TIP.2003.819861.

[6] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2018, pp. 586–595. doi: 10.1109/CVPR.2018.00068.

[7] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards Deep Learning Models Resistant to Adversarial Attacks,” in Proc. Int. Conf. Learning Representations, 2018. Available: https://arxiv.org/abs/1706.06083